1. Your organization has an API Gateway that fronts dozens of microservices. How would you design authentication and authorization at the gateway layer using Spring Security (or Spring Cloud Gateway) before traffic hits individual services?

A: I would design the gateway as an OAuth2 Resource Server to handle authentication and initial authorization. A custom JWT filter in the gateway's security chain intercepts the token from the client, validates its signature, and checks core scopes. The gateway then propagates a cleaned or new, internally signed Gateway Token (containing essential user roles) to the downstream services. This enforces policies centrally and offloads validation work from the microservices.

2. How would you implement dynamic access-control policies in Spring Security so that authorization rules can change without redeployment?

A: I would implement dynamic policies using Spring Expression Language (SpEL) and custom AccessDecisionVoters (or modern equivalents). The AccessDecisionVoters read rules from a centralized location (like a database or a configuration server) instead of hardcoding them. The @PreAuthorize annotation would then call these custom voters, allowing the authorization logic to be updated dynamically without requiring a full application redeployment.

3. You are designing the security architecture for a banking REST API. What end-to-end controls would you apply in Spring Boot?

A: I would start with Threat Modeling to identify risks.

- Auth/TLS: Enforce OAuth2/JWT authentication and mTLS (Mutual TLS) for all service communication.
- Rate Limiting: Apply strict rate limits at the gateway to prevent high-volume attacks.
- Secrets: Use a Secret Manager (like Vault) for database passwords and encryption keys.
- Audit/Trace: Implement Distributed Tracing (Micrometer) and detailed Audit Logging for all financial transactions, including fraud detection signals.

4. In a distributed microservices ecosystem, how would you secure service-to-service communication so services trust each other only through verifiable identity?

A: I would implement Mutual TLS (mTLS) where every service verifies the identity of the service it is communicating with via cryptographic certificates. This creates a zero-trust network where service identity, rather than network location, is the basis of trust. Tools like

Istio in a service mesh automate the generation and rotation of these certificates (workload identity) at scale.

5. How would you integrate a centralized authorization system (OPA, Keycloak authorization services, Cedar, AWS IAM-based policies) with Spring Security so that your microservices use external policy decisions instead of hardcoded role checks?

A: I would integrate the external system (e.g., OPA) by creating a custom Spring Security Filter or AccessDecisionVoter that acts as a Policy Enforcement Point (PEP). This PEP intercepts the request and sends the necessary context (user, resource path, action) to the external Policy Decision Point (PDP) via HTTP. The microservice then grants access based on the simple ALLOW or DENY decision returned by the external system.

6. What strategy would you use to design secret rotation and secure key management for tokens, DB passwords, certificate bundles, and encryption keys in a Spring microservices architecture?

A: I would use a centralized Secret Management platform like HashiCorp Vault. All secrets are stored and encrypted there. I would configure the Spring Boot applications to automatically read and refresh these secrets via a client library at runtime, enabling automated rotation. This ensures that passwords, tokens, and certificates are regularly updated without human intervention or application redeployment.

7. Your microservices run in multiple environments and clouds. How would you design a zero-trust, identity-based, environment-agnostic security model using Spring Security?

A: I would design a model that assumes no implicit trust (zero-trust).

Identity: Use a unified identity provider (e.g., Okta/Keycloak) to issue JWTs based on verifiable workload identity (mTLS certificates).

Enforcement: Every service would run with strict Network Policies (denying all) and use Spring Security's OAuth2 Resource Server to verify every incoming token/certificate.

Agnosticism: All cloud- or environment-specific secrets are managed centrally and injected via environment variables, keeping the application code identical across all deployment targets.